

Partie 7 : Authentication & Autorisation

Définition:

Un middleware est un filtre qui peut contrôler, modifier ou bloquer une requête avant qu'elle arrive au contrôleur, ou agir sur la réponse avant qu'elle ne soit envoyée au client.

Rôle principal :

- ✓ Sécurité : vérifier si l'utilisateur est authentifié (auth)
- ✓ Filtrage : restreindre l'accès selon le rôle (admin)
- ✓ Modification de la requête : ajouter des headers, log, ou validation
- ✓ Transformation de la réponse : compresser les données, ajouter des headers

php artisan make:middleware CheckAge

```
class CheckAge
{
    /**
     * Handle an incoming request.
     */
    public function handle(Request $request, Closure $next)
    {
        if ($request->age < 18) {
            // Bloque la requête et redirige
            return redirect('home');
        }

        // Sinon laisse passer la requête
        return $next($request);
    }
}
```

Définition:

Un middleware est un filtre qui peut contrôler, modifier ou bloquer une requête avant qu'elle arrive au contrôleur, ou agir sur la réponse avant qu'elle ne soit envoyée au client.

Rôle principal :

- ✓ Sécurité : vérifier si l'utilisateur est authentifié (auth)
- ✓ Filtrage : restreindre l'accès selon le rôle (admin)
- ✓ Modification de la requête : ajouter des headers, log, ou validation
- ✓ Transformation de la réponse : compresser les données, ajouter des headers

php artisan make:middleware CheckAge

```
use App\Http\Middleware\CheckAge;

Route::get('/restricted', function () {
    return "Access granted";
})->middleware(CheckAge::class); // ← pas d'alias, juste la classe
```

Définition:

Un middleware est un filtre qui peut contrôler, modifier ou bloquer une requête avant qu'elle arrive au contrôleur, ou agir sur la réponse avant qu'elle ne soit envoyée au client.

Rôle principal :

- ✓ Sécurité : vérifier si l'utilisateur est authentifié (auth)
- ✓ Filtrage : restreindre l'accès selon le rôle (admin)
- ✓ Modification de la requête : ajouter des headers, log, ou validation
- ✓ Transformation de la réponse : compresser les données, ajouter des headers

php artisan make:middleware CheckAge

```
Route::middleware([CheckAge::class])->group(function () {  
  
    Route::get('/dashboard', function () {  
        return "Dashboard";  
    });  
  
    Route::get('/profile', function () {  
        return "Profile";  
    });  
  
});
```

7. Authentication & Autorisation

Authentication dans Laravel

Définition:

L'authentification permet de vérifier l'identité de l'utilisateur.

Laravel propose des packages prêts à l'emploi :

- **Laravel Breeze** ⇒ simple, minimaliste, supporte Blade ou Inertia
- **Laravel UI** ⇒ avec Bootstrap / Vue / React
- **Jetstream** ⇒ pour projets plus avancés avec Teams

```
composer require laravel/breeze --dev
php artisan breeze:install
npm install && npm run dev
php artisan migrate
```

Routes:

/login
/register
/dashboard
/logout

Controllers:

AuthenticatedSessionController
RegisteredUserController
PasswordResetController

Middleware (déjà prêt)

```
Route::get('/dashboard', function () {
    return view('dashboard');
})->middleware(['auth']);
```

Vérifier utilisateur connecté

```
@if (Auth::check())
    <p>Welcome {{ Auth::user()->name }}</p>
@endif
```

Views Blade:

resources/views/auth/login.blade.php
resources/views/auth/register.blade.php
resources/views/dashboard.blade.php

Définition:

Après le login (authentication), on doit vérifier :



“Est-ce que cet utilisateur a le droit de faire cette action ?”

Exemple :

- un **admin** peut supprimer des posts
- un **user normal** ne peut pas

Méthode simple (Middleware + role)

Ajouter un champ role dans users

```
$table->string('role')->default('user');
```

Middleware simple

```
php artisan make:middleware RoleMiddleware
```

Code Middleware

```
public function handle($request, Closure $next, $role)
{
    if (!auth()->check() || auth()->user()->role !== $role) {
        abort(403);
    }

    return $next($request);
}
```

Enregistrement

```
'role' => \App\Http\Middleware\RoleMiddleware::class,
```

Utilisation

```
Route::get('/admin', function () {
    return "Admin area";
})->middleware('role:admin');
```

7. Authentication & Autorisation

Mini TP :

Objectif :

- Mettre en place l'authentification avec Laravel Breeze
- Gérer les rôles utilisateurs (admin / user)
- Protéger les routes avec middleware

Etapas :

- Installer Breeze avec Blade
- Configurer l'authentification complète (login, register, logout).
- Tester l'accès aux routes ***/login, /register*** et ***/dashboard***.
- Ajouter un champ role dans la table users.
- Créer deux utilisateurs : ***admin*** et ***user***
- Créer un middleware ***RoleMiddleware***
- Protéger la route ***/admin*** pour qu'elle soit accessible uniquement aux administrateurs.